

字符串算法与字符串数据结构

Nesraychan

Tsinghua University

July 30, 2026

字符串问题，是什么？

字符串算法处理的，本质上是“相等关系”：两个前缀是否相等，一段文本在哪里出现，两个后缀的公共前缀有多长。

字符串问题，是什么？

字符串算法处理的，本质上是“相等关系”：两个前缀是否相等，一段文本在哪里出现，两个后缀的公共前缀有多长。

单个模式串，我们有 KMP、Z 函数；多个模式串，我们有 Trie 与 AC 自动机；需要研究所有后缀，我们有 SA；需要研究所有子串，我们有 SAM。

字符串问题，是什么？

字符串算法处理的，本质上是“相等关系”：两个前缀是否相等，一段文本在哪里出现，两个后缀的公共前缀有多长。

单个模式串，我们有 KMP、Z 函数；多个模式串，我们有 Trie 与 AC 自动机；需要研究所有后缀，我们有 SA；需要研究所有子串，我们有 SAM。

这节课不准备把这些东西当成六块互不相干的模板。更重要的问题是：一个算法到底压缩了哪些状态，又保留了哪些足以完成计数、匹配与优化的信息。

Trie

Trie 把字符串集合中的公共前缀合并。一个节点对应一个前缀，一条边对应在末尾加入一个字符。

Trie

Trie 把字符串集合中的公共前缀合并。一个节点对应一个前缀，一条边对应在末尾加入一个字符。

于是，插入和查询一个长度为 m 的字符串都是 $O(m)$ ；总节点数不超过所有字符串长度之和。

Trie

Trie 把字符串集合中的公共前缀合并。一个节点对应一个前缀，一条边对应在末尾加入一个字符。

于是，插入和查询一个长度为 m 的字符串都是 $O(m)$ ；总节点数不超过所有字符串长度之和。

但这只是最基础的说法。竞赛中更重要的是：Trie 是一棵由“前缀状态”构成的树，所以树上 dp、dfs 序、可持久化、01-Trie 上的贪心，都可以自然地搬过来。

01-Trie 与异或

把一个整数写成定长二进制串插入 Trie，就得到了 01-Trie。

01-Trie 与异或

把一个整数写成定长二进制串插入 Trie，就得到了 01-Trie。

询问 $\max_{y \in S}(x \oplus y)$ 时，从高位到低位尽量走与 x 当前位不同的边；询问最小值则尽量走相同的边。因为高位的贡献严格支配所有低位，所以这个贪心是正确的。

01-Trie 与异或

把一个整数写成定长二进制串插入 Trie，就得到了 01-Trie。

询问 $\max_{y \in S}(x \oplus y)$ 时，从高位到低位尽量走与 x 当前位不同的边；询问最小值则尽量走相同的边。因为高位的贡献严格支配所有低位，所以这个贪心是正确的。

更复杂的题通常不再询问单个数，而是同时递归两棵 Trie 子树，统计两组数之间的最小异或、异或值域中的数对数量，或者数对异或和。

[CF888G] Xor-MST

给定 n 个非负整数，把每个整数看作一个点。任意两点 i, j 之间都有一条权值为

$$a_i \oplus a_j$$

的无向边。

求这张完全图的最小生成树边权和。

$$1 \leq n \leq 2 \times 10^5, \quad 0 \leq a_i < 2^{30}.$$

[CF888G] Xor-MST

按最高位把数分成 0/1 两组。组内边这一位为 0，组间边这一位为 1，所以如果两组均非空，MST 一定先分别连通两组，再选择恰好一条最小的跨组边。

[CF888G] Xor-MST

按最高位把数分成 0/1 两组。组内边这一位为 0，组间边这一位为 1，所以如果两组均非空，MST 一定先分别连通两组，再选择恰好一条最小的跨组边。

于是递归处理两组。若两组均非空，还必须加入一条跨组边；把其中一组的所有数插入 01-Trie，逐个查询另一组，即可求出最小跨组异或值。

[CF888G] Xor-MST

按最高位把数分成 0/1 两组。组内边这一位为 0，组间边这一位为 1，所以如果两组均非空，MST 一定先分别连通两组，再选择恰好一条最小的跨组边。

于是递归处理两组。若两组均非空，还必须加入一条跨组边；把其中一组的所有数插入 01-Trie，逐个查询另一组，即可求出最小跨组异或值。

一个点会暴力查询 $O(\log V)$ 次，单次查询复杂度 $O(\log V)$ ，所以总复杂度为 $O(n \log^2 V)$ 。

[CF241B] Friends

给定 n 个非负整数。对每个无序对 $i < j$ 定义权值 $a_i \oplus a_j$ 。

从全部 $\binom{n}{2}$ 个数对中选出权值最大的 k 个，求它们的权值之和，对 $10^9 + 7$ 取模。

$$n \leq 5 \times 10^4, \quad 0 \leq a_i \leq 10^9, \quad 1 \leq k \leq \binom{n}{2}.$$

[CF241B] Friends

先把所有数插入 01-Trie。现在固定 x ，计算有多少个 y 满足 $x \oplus y \geq X$ 。从最高位向下比较：如果 X 的当前位是 0，那么让异或结果的当前位取 1 后，无论低位是什么都已经满足条件，可以直接加入这一子树的大小；另一条分支仍需继续比较。如果 X 的当前位是 1，异或结果的当前位也只能取 1。一次查询需要 $O(\log A)$ 时间。

[CF241B] Friends

先把所有数插入 01-Trie。现在固定 x ，计算有多少个 y 满足 $x \oplus y \geq X$ 。从最高位向下比较：如果 X 的当前位是 0，那么让异或结果的当前位取 1 后，无论低位是什么都已经满足条件，可以直接加入这一子树的大小；另一条分支仍需继续比较。如果 X 的当前位是 1，异或结果的当前位也只能取 1。一次查询需要 $O(\log A)$ 时间。

对全部 $x = a_i$ 求和得到有序对数量 $C(X)$ 。删去可能的 $i = j$ ，再除以 2，就是无序对数量。二分最大的 X 使 $C(X) \geq k$ ；设严格大于 X 的对数为 c 、权值和为 s ，答案为

$$s + (k - c)X.$$

[CF241B] Friends: 题解（续）

求 s 时，在 Trie 每个节点维护子树内各二进制位为 1 的个数。某棵子树被整体计入后，可在 $O(\log A)$ 内按位计算 $\sum(x \oplus y)$ 。

[CF241B] Friends: 题解（续）

求 s 时，在 Trie 每个节点维护子树内各二进制位为 1 的个数。某棵子树被整体计入后，可在 $O(\log A)$ 内按位计算 $\sum(x \oplus y)$ 。

于是我们做完了。时间复杂度 $O(n \log^2 A)$ 。

Manacher

朴素地枚举回文中心并向两侧扩展是 $O(n^2)$ 的。Manacher 的关键是维护当前右端点最远的回文区间 $[l, r]$ 。

Manacher

朴素地枚举回文中心并向两侧扩展是 $O(n^2)$ 的。Manacher 的关键是维护当前右端点最远的回文区间 $[l, r]$ 。

计算新中心 i 时，若 $i \leq r$ ，可以先利用它关于区间中心的对称点，得到一个必然正确的初始半径；之后只比较尚未覆盖的字符。

Manacher

朴素地枚举回文中心并向两侧扩展是 $O(n^2)$ 的。Manacher 的关键是维护当前右端点最远的回文区间 $[l, r]$ 。

计算新中心 i 时，若 $i \leq r$ ，可以先利用它关于区间中心的对称点，得到一个必然正确的初始半径；之后只比较尚未覆盖的字符。

每次暴力比较成功都会让 r 增大，所以总复杂度为 $O(n)$ 。
统一插入分隔符可以同时处理奇回文和偶回文。

Manacher 能做什么？

Manacher 给出的不是所有回文子串，而是每个中心的最大半径；其余回文由“半径缩短”隐含表示。

Manacher 能做什么？

Manacher 给出的不是所有回文子串，而是每个中心的最大半径；其余回文由“半径缩短”隐含表示。

因此，判断某区间是否回文、统计按长度分类的回文、寻找覆盖某点的最长回文，都可以离线完成。

Manacher 能做什么？

Manacher 给出的不是所有回文子串，而是每个中心的最大半径；其余回文由“半径缩短”隐含表示。

因此，判断某区间是否回文、统计按长度分类的回文、寻找覆盖某点的最长回文，都可以离线完成。

但如果问题关心的是不同回文串之间的后缀关系，或者需要在线加入字符，Manacher 就不够自然了。这时应当考虑回文自动机。

回文自动机 (PAM)

每个节点表示一个本质不同的回文串; $fail_u$ 指向 u 的最长真回文后缀; 从 u 经过字符 c 的转移, 表示在两侧同时加入 c 。

回文自动机 (PAM)

每个节点表示一个本质不同的回文串; $fail_u$ 指向 u 的最长真回文后缀; 从 u 经过字符 c 的转移, 表示在两侧同时加入 c 。

加入一个新字符时, 从上一次的最长回文后缀开始跳 fail, 找到第一个能够向两侧扩展的节点。若转移不存在, 就新建一个回文串节点。

回文自动机 (PAM)

每个节点表示一个本质不同的回文串; $fail_u$ 指向 u 的最长真回文后缀; 从 u 经过字符 c 的转移, 表示在两侧同时加入 c 。

加入一个新字符时, 从上一次的最长回文后缀开始跳 fail, 找到第一个能够向两侧扩展的节点。若转移不存在, 就新建一个回文串节点。

长度为 -1 、 0 的两个虚根可以统一奇偶性与边界。节点数至多 $n + 2$; 定长字符集下每次转移为 $O(1)$, 建树总复杂度 $O(n)$ 。

[CF932G] Palindrome Partition

给定一个偶数长度的字符串 S 。把它划分成偶数个非空连续段

$$T_1, T_2, \dots, T_{2k},$$

要求对所有 i , 都有 $T_i = T_{2k-i+1}$ 。

求合法划分方案数, 对 $10^9 + 7$ 取模。

$$2 \leq |S| \leq 10^6。$$

[CF932G] Palindrome Partition

把原串首尾交替重排为

$$S_1, S_n, S_2, S_{n-1}, \dots$$

原问题就等价于把新串划分成若干个长度为偶数的回文串。

[CF932G] Palindrome Partition

把原串首尾交替重排为

$$S_1, S_n, S_2, S_{n-1}, \dots$$

原问题就等价于把新串划分成若干个长度为偶数的回文串。

设 dp_i 为重排串前 i 位的方案数, $dp_0 = 1$ 。对偶数 i ,

$$dp_i = \sum_{\substack{u \text{ 是前缀 } i \text{ 的回文后缀} \\ len_u \text{ 为偶数}}} dp_{i-len_u};$$

奇数 i 令 $dp_i = 0$ 。

[CF932G] Palindrome Partition: 题解 (续)

根据 Border 理论, 所有大于一半的 Border 构成等差数列, 而回文 Border 也有这样的性质。所以总共是 $O(\log n)$ 段等差数列。

[CF932G] Palindrome Partition: 题解 (续)

根据 Border 理论, 所有大于一半的 Border 构成等差数列, 而回文 Border 也有这样的性质。所以总共是 $O(\log n)$ 段等差数列。

对于公差为 D 的同一段等差数列, f_i 和 f_{i-D} 的转移只差了 $O(1)$ 个元素。

[CF932G] Palindrome Partition: 题解 (续)

根据 Border 理论, 所有大于一半的 Border 构成等差数列, 而回文 Border 也有这样的性质。所以总共是 $O(\log n)$ 段等差数列。

对于公差为 D 的同一段等差数列, f_i 和 f_{i-D} 的转移只差了 $O(1)$ 个元素。

所以我们可以记录当前每一个等差数列的贡献, 这样转移的时候只需要处理总共的 $O(\log n)$ 个增量。于是总时间复杂度 $O(n \log n)$ 。

[nikkei2019-2-final] 逆にする函数

对函数 $f: \{1, \dots, m\} \rightarrow \{1, \dots, m\}$, 若序列 $(v_1, \dots, v_k)$ 满足 $f(v_i) = v_{k-i+1}$, 则称 f 可以逆转该序列。

给定长度为 n 的序列 a 。对于它的每个非空连续子数组, 统计能够逆转该子数组的函数数量, 并求这些数量之和, 答案对 998244353 取模。

若两个函数在至少一个位置的函数值不同, 则视为不同函数。

$$1 \leq n, m \leq 3 \times 10^5, 1 \leq a_i \leq m.$$

[nikkei2019-2-final] 逆にする函数

对一个区间，假设存在一组解，那么解的组数肯定是 m 的自由元个数次方。

[nikkei2019-2-final] 逆にする函数

对一个区间，假设存在一组解，那么解的组数肯定是 m 的自由元个数次方。

我们又发现，这个是否有解的判定和回文串很相似，进而我们考虑 Manacher 算法。

[nikkei2019-2-final] 逆にする函数

对一个区间，假设存在一组解，那么解的组数肯定是 m 的自由元个数次方。

我们又发现，这个是否有解的判定和回文串很相似，进而我们考虑 Manacher 算法。

对每个回文中心求解，每次拓展/缩短区间，我们只需要判断当前区间还有没有这个颜色，就可以处理自由元个数，进而 $O(1)$ 增加/扣除答案和。

[nikkei2019-2-final] 逆にする函数

对一个区间，假设存在一组解，那么解的组数肯定是 m 的自由元个数次方。

我们又发现，这个是否有解的判定和回文串很相似，进而我们考虑 Manacher 算法。

对每个回文中心求解，每次拓展/缩短区间，我们只需要判断当前区间还有没有这个颜色，就可以处理自由元个数，进而 $O(1)$ 增加/扣除答案和。

由经典结论，缩短区间的暴力次数也是 $O(n)$ 的，所以总复杂度 $O(n)$ 。

扩展 KMP: Z 函数

对字符串 S , 定义 $z_i = LCP(S, S[i \dots n])$ 。这就是每个后缀与整个字符串的最长公共前缀。

扩展 KMP: Z 函数

对字符串 S , 定义 $z_i = LCP(S, S[i \dots n])$ 。这就是每个后缀与整个字符串的最长公共前缀。

算法维护最靠右的匹配段 $[l, r]$ 。若 $i \leq r$, 先令

$$z_i \leftarrow \min(z_{i-l}, r - i + 1),$$

再从 $r + 1$ 之后继续比较。

扩展 KMP: Z 函数

对字符串 S , 定义 $z_i = LCP(S, S[i \dots n])$ 。这就是每个后缀与整个字符串的最长公共前缀。

算法维护最靠右的匹配段 $[l, r]$ 。若 $i \leq r$, 先令

$$z_i \leftarrow \min(z_{i-l}, r - i + 1),$$

再从 $r + 1$ 之后继续比较。

与 Manacher 一样, 每次额外比较成功都会推动右端点, 所以总复杂度是 $O(n)$ 。用同样的算法可以, 求 S 与 T 每个后缀的 LCP, 这通常称作扩展 KMP。

[CF1051E] Vasya and Big Integers

给定三个没有前导零的十进制大整数串 A, L, R 。把 A 划分成若干段，要求每段作为整数均属于 $[L, R]$ ，并且除数字 0 本身外不得含有前导零。

求合法划分方案数，对 998244353 取模。

$$|A|, |L|, |R| \leq 10^6。$$

[CF1051E] Vasya and Big Integers

做 exkmp, 长度恰为 $|L|$ 的段是否 $\geq L$ 、长度恰为 $|R|$ 的段是否 $\leq R$, 都可由 LCP 后第一个不同字符在 $O(1)$ 判定。若长度不同, 则更是显然的。

[CF1051E] Vasya and Big Integers

做 exkmp, 长度恰为 $|L|$ 的段是否 $\geq L$ 、长度恰为 $|R|$ 的段是否 $\leq R$, 都可由 LCP 后第一个不同字符在 $O(1)$ 判定。若长度不同, 则更是显然的。

若 $A_i \neq 0$, 合法长度是一个整数区间: 下端为 $|L|$ 或 $|L| + 1$, 上端为 $|R|$ 或 $|R| - 1$; 再与剩余长度相交。若 $A_i = 0$, 唯一可能的段是单个字符 “0”, 且仅当 $L \leq 0 \leq R$ 。

[CF1051E] Vasya and Big Integers

做 exkmp, 长度恰为 $|L|$ 的段是否 $\geq L$ 、长度恰为 $|R|$ 的段是否 $\leq R$, 都可由 LCP 后第一个不同字符在 $O(1)$ 判定。若长度不同, 则更是显然的。

若 $A_i \neq 0$, 合法长度是一个整数区间: 下端为 $|L|$ 或 $|L| + 1$, 上端为 $|R|$ 或 $|R| - 1$; 再与剩余长度相交。若 $A_i = 0$, 唯一可能的段是单个字符 “0”, 且仅当 $L \leq 0 \leq R$ 。

则可以直接 dp, 主动转移, 可以直接维护差分数组。则时间复杂度 $O(n)$ 。

[FJOI2022] 重复函数

给定长度为 n 的小写字母字符串 S 。

定义 $f_S(i)$ 为最长的字符串 A ，使得 $S = A\#A\#\cdots\#A$ ，其中 A 恰好出现 i 次，每个 $\#$ 均可替换为任意字符串（包括空串）。若不存在这样的非空字符串，则 $f_S(i)$ 为空串。

对所有 $2 \leq i \leq n$ ，求 $f_S(i)$ 及其长度 $|f_S(i)|$ 。

$$|S| \leq 10^6.$$

[FJOI2022] 重复函数

从小到大枚举 A 。 A 能在 p 开头的地方开始的充要条件为， $lcp_p \geq |A|$ ，所以一定是从某个时刻开始就不行了。

[FJOI2022] 重复函数

从小到大枚举 A 。 A 能在 p 开头的地方开始的充要条件为， $lcp_p \geq |A|$ ，所以一定是从某个时刻开始就不行了。

而我们发现，对于一个确定的 A 一定是贪心的填最优，这很显然。

[FJOI2022] 重复函数

从小到大枚举 A 。 A 能在 p 开头的地方开始的充要条件为， $lcp_p \geq |A|$ ，所以一定是从某个时刻开始就不行了。

而我们发现，对于一个确定的 A 一定是贪心的填最优，这很显然。

所以我们使用并查集维护 i 的下一个可行位置，如果 p 变得不行，就和 $p + 1$ 合并。

[FJOI2022] 重复函数

从小到大枚举 A 。 A 能在 p 开头的地方开始的充要条件为， $lcp_p \geq |A|$ ，所以一定是从某个时刻开始就不行了。

而我们发现，对于一个确定的 A 一定是贪心的填最优，这很显然。

所以我们使用并查集维护 i 的下一个可行位置，如果 p 变得不行，就和 $p + 1$ 合并。

显然一次查询最多跳 $\frac{|S|}{|A|}$ 次，于是使用线性并查集复杂度即为 $O(|S| \log |S|)$ 。

AC 自动机

Trie 能共享模式串的前缀，但文本匹配失败时，我们希望跳到当前串的最长可用后缀。于是像 KMP 一样，为每个 Trie 节点建立 fail 指针。

AC 自动机

Trie 能共享模式串的前缀，但文本匹配失败时，我们希望跳到当前串的最长可用后缀。于是像 KMP 一样，为每个 Trie 节点建立 fail 指针。

沿 Trie 做 BFS。节点 u 经字符 c 的失配转移，应当继承 $fail_u$ 经 c 的转移；不存在的 Trie 边也补成这个转移，就得到一个完整 DFA。

AC 自动机

Trie 能共享模式串的前缀，但文本匹配失败时，我们希望跳到当前串的最长可用后缀。于是像 KMP 一样，为每个 Trie 节点建立 fail 指针。

沿 Trie 做 BFS。节点 u 经字符 c 的失配转移，应当继承 $fail_u$ 经 c 的转移；不存在的 Trie 边也补成这个转移，就得到一个完整 DFA。

匹配文本时，每个字符只做一次自动机转移。到达节点 u ，意味着 fail 树上从根到 u 路径对应的所有模式串都在当前位置结尾。

[CF1437G] Death DBMS

给定 n 个模式串，每个模式串有一个可修改的权值，初始均为 0。支持：

- 把第 i 个模式串的权值修改为 x ；
- 给定文本串 T ，求所有在 T 中出现的模式串的最大权值；若一个都没有，输出 -1 。

$n, q \leq 3 \times 10^5$ ，所有字符串总长不超过 3×10^5 。

[CF1437G] Death DBMS

对所有模式串建 AC 自动机。文本走到状态 u 时，所有匹配到的模式串终止节点，恰好是 fail 树上根到 u 的祖先。

[CF1437G] Death DBMS

对所有模式串建 AC 自动机。文本走到状态 u 时，所有匹配到的模式串终止节点，恰好是 fail 树上根到 u 的祖先。

那么原问题相当于，树上每个点有一个权值，支持修改。你要求若干次点到根路径的最大值。

[CF1437G] Death DBMS

对所有模式串建 AC 自动机。文本走到状态 u 时，所有匹配到的模式串终止节点，恰好是 fail 树上根到 u 的祖先。

那么原问题相当于，树上每个点有一个权值，支持修改。你要求若干次点到根路径的最大值。

这是一个经典的问题，使用树剖 + 线段树的话，复杂度为 $O(n \log^2 n)$ 。使用全局平衡二叉树的话，时间复杂度为 $O(n \log n)$ 。

[CF590E] Birthday

给定 n 个互不相同的 01 字符串。选择尽量多的字符串，要求任意一个被选字符串都不是另一个被选字符串的子串；同时输出一种最优选择。

$n \leq 750$ ，所有字符串总长不超过 10^7 。

[CF590E] Birthday

首先建立包含关系：当且仅当 S_i 是 S_j 的子串时，记作 $i \prec j$ 。

[CF590E] Birthday

首先建立包含关系：当且仅当 S_i 是 S_j 的子串时，记作 $i \prec j$ 。

用 AC 自动机扫描 S_j ；每到达一个状态，它在 fail 链上的终止节点就是已经出现的模式串。

[CF590E] Birthday

首先建立包含关系：当且仅当 S_i 是 S_j 的子串时，记作 $i \prec j$ 。

用 AC 自动机扫描 S_j ；每到达一个状态，它在 fail 链上的终止节点就是已经出现的模式串。

直接遍历整条 fail 链会超时，因此在扫描同一个 S_j 时，我们找到在 fail 树上最近的祖先，由于有传递关系，所以我们最后只要跑一遍传递闭包就好了。

[CF590E] Birthday

首先建立包含关系：当且仅当 S_i 是 S_j 的子串时，记作 $i \prec j$ 。

用 AC 自动机扫描 S_j ；每到达一个状态，它在 fail 链上的终止节点就是已经出现的模式串。

直接遍历整条 fail 链会超时，因此在扫描同一个 S_j 时，我们找到在 fail 树上最近的祖先，由于有传递关系，所以我们最后只要跑一遍传递闭包就好了。

然后我们要求最长反链，这是网络流的经典问题。于是时间复杂度 $O(\sum |S_i| + \frac{n^3}{\omega} + n^{2.5})$ 。

[CF587F] Duff is Mad

给定 n 个字符串 $S_1, \dots, S_n$ 。每次询问给出 l, r, k , 求

$$\sum_{i=l}^r occ(S_i, S_k),$$

其中重叠出现需要重复计算。

$n, q \leq 10^5$, 所有字符串总长不超过 10^5 。

[CF587F] Duff is Mad

对所有 S_i 建立 AC 自动机，并把 S_i 的终止节点记为 pos_i 。扫描文本 S_k 时，每到达一个状态 u ，此时匹配成功的模式串正好对应 fail 树上 u 的祖先。因此，原询问等价于：对扫描 S_k 经过的每个状态，统计编号在 $[l, r]$ 内的模式串中，有多少个终止节点是它的祖先。

[CF587F] Duff is Mad

对所有 S_i 建立 AC 自动机，并把 S_i 的终止节点记为 pos_i 。扫描文本 S_k 时，每到达一个状态 u ，此时匹配成功的模式串正好对应 fail 树上 u 的祖先。因此，原询问等价于：对扫描 S_k 经过的每个状态，统计编号在 $[l, r]$ 内的模式串中，有多少个终止节点是它的祖先。

下面按照文本长度 $|S_k|$ 分类处理。对于 $|S_k| > B$ 的长文本，不同的 k 至多有 N/B 个。固定一个长文本，在 AC 自动机上匹配一次，并按 fail 深度从大到小累加各状态的访问次数。累加完成后，有

$$cnt[pos_i] = occ(S_i, S_k).$$

再按照模式串编号求前缀和，就能在 $O(1)$ 时间内回答与该文本有关的询问。

[CF587F] Duff is Mad: 题解 (续)

对于短文本，定义

$$F(x, k) = \sum_{i=1}^x \text{occ}(S_i, S_k),$$

于是一次询问等于 $F(r, k) - F(l - 1, k)$ 。按照 x 从小到大离线处理。加入模式串 S_x 时，将其终止节点的 fail 子树整体加一。计算 $F(x, k)$ 时，匹配长度不超过 B 的文本 S_k ，并查询沿途各状态的点值。

[CF587F] Duff is Mad: 题解 (续)

对于短文本，定义

$$F(x, k) = \sum_{i=1}^x \text{occ}(S_i, S_k),$$

于是一次询问等于 $F(r, k) - F(l-1, k)$ 。按照 x 从小到大离线处理。加入模式串 S_x 时，将其终止节点的 fail 子树整体加一。计算 $F(x, k)$ 时，匹配长度不超过 B 的文本 S_k ，并查询沿途各状态的点值。

所以我们有 $O(n)$ 次区间加， $O(nB)$ 次单点查询。

[CF587F] Duff is Mad: 题解 (续)

对于短文本，定义

$$F(x, k) = \sum_{i=1}^x \text{occ}(S_i, S_k),$$

于是一次询问等于 $F(r, k) - F(l-1, k)$ 。按照 x 从小到大离线处理。加入模式串 S_x 时，将其终止节点的 fail 子树整体加一。计算 $F(x, k)$ 时，匹配长度不超过 B 的文本 S_k ，并查询沿途各状态的点值。

所以我们有 $O(n)$ 次区间加， $O(nB)$ 次单点查询。

取 $B = \sqrt{N}$ ，并使用分块完成上面的任务，复杂度就是 $O(N\sqrt{N})$ 的。

后缀数组 SA

将所有后缀按字典序排序, sa_i 表示排名第 i 的后缀起点, rk_i 表示后缀 i 的排名。

后缀数组 SA

将所有后缀按字典序排序, sa_i 表示排名第 i 的后缀起点, rk_i 表示后缀 i 的排名。

倍增算法每轮按二元组

$$(rk_i, rk_{i+2^k})$$

排序, 并重新离散化排名。用计数排序可做到 $O(n \log n)$ 。

后缀数组 SA

将所有后缀按字典序排序, sa_i 表示排名第 i 的后缀起点, rk_i 表示后缀 i 的排名。

倍增算法每轮按二元组

$$(rk_i, rk_{i+2^k})$$

排序, 并重新离散化排名。用计数排序可做到 $O(n \log n)$ 。

$height_i = LCP(sa_{i-1}, sa_i)$ 。利用相邻后缀删除首字符后 LCP 至少减少一的性质, 可以在线性时间求出整个 height 数组。

height 数组，是什么？

不妨先交换使 $rk_i < rk_j$ 。两个任意后缀 i, j 的 LCP，等于它们排名之间 height 的最小值：

$$LCP(i, j) = \min_{rk_i < t \leq rk_j} height_t.$$

height 数组，是什么？

不妨先交换使 $rk_i < rk_j$ 。两个任意后缀 i, j 的 LCP，等于它们排名之间 height 的最小值：

$$LCP(i, j) = \min_{rk_i < t \leq rk_j} height_t.$$

所以加上 RMQ 后，任意两后缀 LCP 可以 $O(1)$ 查询。

height 数组，是什么？

不妨先交换使 $rk_i < rk_j$ 。两个任意后缀 i, j 的 LCP，等于它们排名之间 height 的最小值：

$$LCP(i, j) = \min_{rk_i < t \leq rk_j} height_t.$$

所以加上 RMQ 后，任意两后缀 LCP 可以 $O(1)$ 查询。

另一方面，若要求至少 k 个后缀拥有长度至少 L 的公共前缀，只需看 height 数组中是否存在长度 $k-1$ 、最小值至少 L 的连续段。许多“重复子串”问题都因此变成 height 上的滑动窗口或单调栈问题。

[NOI2015] 品酒大会

给定一个长度为 n 的字符串 S ，以及每个位置的权值 a_i 。对于每个 $r = 0, 1, \dots, n-1$ ，考虑所有不同位置对 (i, j) ，满足后缀 $S[i \dots n]$ 与 $S[j \dots n]$ 的 LCP 至少为 r 。

求这样的无序位置对数量，以及这些位置对中 $a_i a_j$ 的最大值。若不存在位置对，最大值输出 0。

$$n \leq 3 \times 10^5, |a_i| \leq 10^9。$$

[NOI2015] 品酒大会

从大到小枚举 r ，那么 r 减小的时候，后缀排序后的一些位置会被合并起来。

[NOI2015] 品酒大会

从大到小枚举 r ，那么 r 减小的时候，后缀排序后的一些位置会被合并起来。

我们使用并查集处理这些合并，第一问答案是每块大小选二之和，第二问是一个块最大值 + 次大值的最大值。

[NOI2015] 品酒大会

从大到小枚举 r ，那么 r 减小的时候，后缀排序后的一些位置会被合并起来。

我们使用并查集处理这些合并，第一问答案是每块大小选二之和，第二问是一个块最大值 + 次大值的最大值。

于是时间复杂度为 $O(n \log n)$ 。

[CF123D] String

对字符串 S 的一个子串 T ，把 T 的所有出现位置按起点排序。令 $F(S, T)$ 为这个出现序列的非空连续子段数量。

求

$$\sum_{T \text{ 是 } S \text{ 的本质不同子串}} F(S, T).$$

$$1 \leq |S| \leq 10^5.$$

[CF123D] String

若 T 出现 c_T 次, 则 $F(S, T) = \frac{c_T(c_T+1)}{2}$.

[CF123D] String

若 T 出现 c_T 次, 则 $F(S, T) = \frac{c_T(c_T+1)}{2}$.

我们把 S 的每个后缀插入 Trie 中。得到的这个树, 每个节点对应的是一个不同子串, 而每个节点的出现次数, 则是子树内叶子的个数。

[CF123D] String

若 T 出现 c_T 次, 则 $F(S, T) = \frac{c_T(c_T+1)}{2}$.

我们把 S 的每个后缀插入 Trie 中。得到的这个树, 每个节点对应的是一个不同子串, 而每个节点的出现次数, 则是子树内叶子的个数。

我们发现这棵树存在很多二度点, 这些点的贡献相同。我们能不能把他们压缩?

[CF123D] String

若 T 出现 c_T 次, 则 $F(S, T) = \frac{c_T(c_T+1)}{2}$.

我们把 S 的每个后缀插入 Trie 中。得到的这个树, 每个节点对应的是一个不同子串, 而每个节点的出现次数, 则是子树内叶子的个数。

我们发现这棵树存在很多二度点, 这些点的贡献相同。我们能不能把他们压缩?

具体来说, 我们可以通过 SA 数组和 height 数组, 把树压缩成 $O(n)$ 个点。

[CF123D] String

若 T 出现 c_T 次, 则 $F(S, T) = \frac{c_T(c_T+1)}{2}$.

我们把 S 的每个后缀插入 Trie 中。得到的这个树, 每个节点对应的是一个不同子串, 而每个节点的出现次数, 则是子树内叶子的个数。

我们发现这棵树存在很多二度点, 这些点的贡献相同。我们能不能把他们压缩?

具体来说, 我们可以通过 SA 数组和 height 数组, 把树压缩成 $O(n)$ 个点。

那么时间复杂度则是 $O(n \log n)$.

[NOI2016] 优秀的拆分

称字符串的一种拆分是优秀的，当且仅当它能写成

$$A + A + B + B,$$

其中 A, B 均为非空字符串。给定若干个字符串，对每个字符串统计其所有子串的优秀拆分方案总数。

字符串长度不超过 3×10^4 ，所有测试串总长不超过 3×10^4 。

[NOI2016] 优秀的拆分

先预处理正串和反串的后缀数组，并用 height 数组建立 RMQ。这样，任意两个后缀的 LCP，以及任意两个前缀向左的 LCS，都能在 $O(1)$ 时间内求出。

[NOI2016] 优秀的拆分

先预处理正串和反串的后缀数组，并用 height 数组建立 RMQ。这样，任意两个后缀的 LCP，以及任意两个前缀向左的 LCS，都能在 $O(1)$ 时间内求出。

下面统计所有平方串 XX 。固定半长 p ，只考察相邻的两个长度为 p 的块，其边界依次取 $p, 2p, 3p, \dots$ 。设边界两侧向左能够匹配的长度为 L ，向右能够匹配的长度为 R ，并将它们分别截断为 $L \leq p - 1$ 和 $R \leq p$ 。

若把平方串的中点从该边界向左移动 t ，两半仍相等当且仅当

$$p - R \leq t \leq L.$$

所以当 $L + R \geq p$ 时，合法中点构成一个连续区间；对“平方串起点”和“平方串终点”分别做区间差分，即得 right_i 与 left_i 。

[NOI2016] 优秀的拆分：题解（续）

每个优秀拆分 $AABB$ 在 AA 与 BB 的交界处唯一计数，因此答案为

$$\sum_{i=1}^{n-1} left_i \cdot right_{i+1}.$$

对于固定的半长 p ，只需检查 $O(n/p)$ 个块边界，因此检查次数之和为

$$\sum_{p=1}^{\lfloor n/2 \rfloor} O(n/p) = O(n \log n).$$

建立后缀数组需要 $O(n \log n)$ 时间，其余统计同样为 $O(n \log n)$ 。

[CF1063F] String Journey

称 $T_1, \dots, T_k$ 为一次 journey, 当 T_i 是 T_{i-1} 的子串且长度严格更短。还要求它们能按从左到右、互不相交的顺序依次嵌入给定字符串 S 。

求 journey 的最大长度, 即最大化 k 。

$$|S| \leq 5 \times 10^5。$$

[CF1063F] String Journey

先使用一个正规化结论：如果存在长度为 k 的 journey，就可以在不改变各段出现位置的前提下适当缩短各段，使第 i 段的长度恰好为 $k - i + 1$ 。于是，相邻两段的长度只差 1，后一段必然是由前一段删除首字符或删除末字符得到的。

[CF1063F] String Journey

先使用一个正规化结论：如果存在长度为 k 的 journey，就可以在不改变各段出现位置的前提下适当缩短各段，使第 i 段的长度恰好为 $k - i + 1$ 。于是，相邻两段的长度只差 1，后一段必然是由前一段删除首字符或删除末字符得到的。

令 f_i 为第一段从 i 开始时的最大链长。判定 $f_i \geq d$ ，等价于存在 $p \geq i + d$ ，满足 $f_p \geq d - 1$ ，且 $S[p \dots p + d - 2]$ 等于

$$S[i \dots i + d - 2] \quad \text{或} \quad S[i + 1 \dots i + d - 1].$$

边界 $p \geq i + d$ 正好保证前两段不相交。

[CF1063F] String Journey: 题解 (续)

要求相当于这两个候选的 rk 得在一个区间里。我们按照 i 从右向左计算, 并建立可持久化线段树: 版本 $root_t$ 包含所有起点 $p \geq t$, 在位置 rk_p 保存 f_p 。因此, 只需在版本 $root_{i+d}$ 的区间中查询最大值是否至少为 $d-1$ 。

[CF1063F] String Journey: 题解 (续)

要求相当于这两个候选的 rk 得在一个区间里。我们按照 i 从右向左计算, 并建立可持久化线段树: 版本 $root_t$ 包含所有起点 $p \geq t$, 在位置 rk_p 保存 f_p 。因此, 只需在版本 $root_{i+d}$ 的区间中查询最大值是否至少为 $d-1$ 。

若长度 d 可行, 删去正规化链的第一段并适当截短即可得到更短可行长度, 所以判定具有单调性, 可以二分 f_i 。每次判定做常数次 SA 区间定位和区间最大值查询, 均为 $O(\log n)$; 总时间 $O(n \log^2 n)$ 、空间 $O(n \log n)$ 。

后缀自动机 SAM

对于一个字符串 S ，我们对其的反串建立后缀树，可以发现本质不同的节点只有 $O(n)$ 个。（即 $endpos$ 集合不同）

后缀自动机 SAM

对于一个字符串 S ，我们对其的反串建立后缀树，可以发现本质不同的节点只有 $O(n)$ 个。（即 $endpos$ 集合不同）

这告诉我们，我们可以把相同集合的元素压缩成一个节点，这样可以得到一个自动机。 $ch_{p,c}$ 表示节点 p 对应的集合的后面加了一个字符 c 所对应的集合。

后缀自动机 SAM

对于一个字符串 S ，我们对其的反串建立后缀树，可以发现本质不同的节点只有 $O(n)$ 个。（即 $endpos$ 集合不同）

这告诉我们，我们可以把相同集合的元素压缩成一个节点，这样可以得到一个自动机。 $ch_{p,c}$ 表示节点 p 对应的集合的后面加了一个字符 c 所对应的集合。

然后我们还要记录 $len_p, fail_p$ 分别表示当前节点对应的最长字符串，和最长的，和自己集合不同的节点的编号。那么本质不同字符串个数就是 $\sum len_p - len_{fail_p}$ 。

后缀自动机 SAM

对于一个字符串 S ，我们对其的反串建立后缀树，可以发现本质不同的节点只有 $O(n)$ 个。（即 $endpos$ 集合不同）

这告诉我们，我们可以把相同集合的元素压缩成一个节点，这样可以得到一个自动机。 $ch_{p,c}$ 表示节点 p 对应的集合的后面加了一个字符 c 所对应的集合。

然后我们还要记录 $len_p, fail_p$ 分别表示当前节点对应的最长字符串，和最长的，和自己集合不同的节点的编号。那么本质不同字符串个数就是 $\sum len_p - len_{fail_p}$ 。

更进一步发现， len 和 $fail$ 的 parent tree，事实上就是后缀树，唯一多的部分就是 ch 的 DAG 结构。

[CF235C] Cyclical Quest

给定一个母串 S ，再给出 m 个询问串 T 。对每次询问，求 S 中有多少个连续子串与 T 循环同构；也就是等于 T 的某个循环移位。

所有字符串均由小写字母组成。

$|S| \leq 10^6$ ，所有询问串长度之和不超过 10^6 。

[CF235C] Cyclical Quest

对 S 建后缀自动机，接着我们让 $T + T$ 在后缀自动机上匹配。

[CF235C] Cyclical Quest

对 S 建后缀自动机，接着我们让 $T + T$ 在后缀自动机上匹配。

那么现在一个子串合法的充要条件，有两个：一是长度 $\leq n$;

[CF235C] Cyclical Quest

对 S 建后缀自动机，接着我们让 $T + T$ 在后缀自动机上匹配。

那么现在一个子串合法的充要条件，有两个：一是长度 $\leq n$ ；
二是其的代表节点，是遍历 $T + T$ 经过的某个点的祖先。

[CF235C] Cyclical Quest

对 S 建后缀自动机，接着我们让 $T + T$ 在后缀自动机上匹配。

那么现在一个子串合法的充要条件，有两个：一是长度 $\leq n$ ；
二是其的代表节点，是遍历 $T + T$ 经过的某个点的祖先。

所以我们可以直接对过程中遍历的 $O(|T|)$ 个关键点建虚树求解，这样复杂度就是 $O(n \log n)$ 的。

给定字符串 S 。每次询问给出 l, r 和模式串 T ，要求找到一个字符串 P ，满足：

- P 是 $S[l \dots r]$ 的子串；
- P 的字典序严格大于 T ；
- 在满足条件的字符串中， P 的字典序最小。

$$|S| \leq 10^5, \quad q, \sum |T| \leq 2 \times 10^5。$$

[CF1037H] Security

这是一个维护 `endpos` 集合的例题。

首先肯定用字典序贪心，所以我们只需要支持快速询问，一个 SAM 上的结点对应的某个长度的字符串，是否在区间 $[l, r]$ 中出现过。

这是一个维护 `endpos` 集合的例题。

首先肯定用字典序贪心，所以我们只需要支持快速询问，一个 SAM 上的结点对应的某个长度的字符串，是否在区间 $[l, r]$ 中出现过。

这相当于存在 `endpos` 在 $[l + len - 1, r]$ 中，这是一个确定的区间。

这是一个维护 `endpos` 集合的例题。

首先肯定用字典序贪心，所以我们只需要支持快速询问，一个 SAM 上的结点对应的某个长度的字符串，是否在区间 $[l, r]$ 中出现过。

这相当于存在 `endpos` 在 $[l + len - 1, r]$ 中，这是一个确定的区间。

对于每个前缀 p ，我们在其对应的节点插入 p 。然后在 `parent tree` 上进行可持久化线段树合并，即可完成我们的目标。

这是一个维护 `endpos` 集合的例题。

首先肯定用字典序贪心，所以我们只需要支持快速询问，一个 SAM 上的结点对应的某个长度的字符串，是否在区间 $[l, r]$ 中出现过。

这相当于存在 `endpos` 在 $[l + len - 1, r]$ 中，这是一个确定的区间。

对于每个前缀 p ，我们在其对应的节点插入 p 。然后在 `parent tree` 上进行可持久化线段树合并，即可完成我们的目标。

时间复杂度 $O(n|\Sigma| \log n)$ 。

[NOI2018] 你的名字

给定母串 S 。每次询问给出字符串 T 和区间 $[l, r]$ ，要求统计 T 有多少个本质不同的子串，没有在 $S[l \dots r]$ 中出现。

$|S|, \sum |T| \leq 5 \times 10^5$ ，询问数不超过 10^5 。

[NOI2018] 你的名字

这个题事实上可以看成前面两个题拼起来。

我们改成计算有多少子串出现过。首先我们使用上一题维护 endpos 的方式，可以用 two-pointer 求出来对于每个右端点 r ，最小的左端点 p_r 使得 $[p_r, r]$ 出现过。

[NOI2018] 你的名字

这个题事实上可以看成前面两个题拼起来。

我们改成计算有多少子串出现过。首先我们使用上一题维护 endpos 的方式，可以用 two-pointer 求出来对于每个右端点 r ，最小的左端点 p_r 使得 $[p_r, r]$ 出现过。

求出所有 $[p_r, r]$ 在 SAM 上的对应节点，那么所有是某个点的严格祖先的节点肯定全出现过。而一个相关节点的出现长度则是一个前缀，所以这是上上个题。

那么做完了，时间复杂度 $O(n|\Sigma| \log n)$ 。

SA 与 SAM，怎么选？

如果问题关心后缀的字典序、相邻后缀 LCP、重复子串的排序结构，SA 往往更直接。

SA 与 SAM, 怎么选?

如果问题关心后缀的字典序、相邻后缀 LCP、重复子串的排序结构, SA 往往更直接。

如果问题关心本质不同子串、在线追加字符、出现次数或在另一个串中匹配, SAM 往往状态更自然。

SA 与 SAM, 怎么选?

如果问题关心后缀的字典序、相邻后缀 LCP、重复子串的排序结构, SA 往往更直接。

如果问题关心本质不同子串、在线追加字符、出现次数或在另一个串中匹配, SAM 往往状态更自然。

两者包含的信息高度接近: SA 用一个后缀排列和 height 数组表达, SAM 用 endpos 等价类表达。选型时不要问“哪个更强”, 而要问“题目的贡献在什么结构上更容易被累加”。

谢谢大家！